

Week 2 Assignment Report – Text Mining

1. Environment and Dataset

This assignment was conducted using Python in Google Colab.

- Python version: **3.13.15**
- PyTorch version: **2.11.0+cpu**
- scikit-learn version: **1.6.1**
- kiwipiepy version: **0.23.2**
- Dataset: Korean review corpus
- Number of documents: **100,000**
- Label format: **0 = negative, 1 = positive**

Each line in the corpus contains one sentiment label and one Korean review sentence. The Count/TF-IDF matrix size was **(100000, 283161)**, and the non-zero ratio was **0.0037%**.

2. Task 1 – Count vs. TF-IDF

Count and TF-IDF were applied to the Korean review corpus.

For the first document:

“노래가 너무 적음”

The output was:

Count Top Terms

1. 적음
2. 노래가
3. 너무

TF-IDF Top Terms

1. 적음
2. 노래가
3. 너무

The notebook also calculated IDF values manually and compared them with scikit-learn:

- 의: $df = 83$, $IDF = 8.0821$
- 는: $df = 89$, $IDF = 8.0131$
- 적음: $df = 33$, $IDF = 8.9866$

The manually calculated IDF values were identical to the scikit-learn results.

Analysis

Count mainly reflects how frequently a word appears in a document, whereas TF-IDF also considers how frequently the word appears across the whole corpus.

The word “적음” appeared in only **33 documents**, so it had a relatively high IDF value of **8.9866**. In comparison, more common words such as 의 and 는 had lower IDF values.

Therefore, TF-IDF gives more importance to words that are relatively rare and document-specific.

The most similar pair among the first 200 documents was document **170** and document **176**, with cosine similarity **0.288**.

3. Task 2 – BM25 Parameter Analysis

The query used for BM25 was:

“노래가 너무”

The default BM25 result returned:

- Score **15.21**, length **3**: 노래가 너무 적음
- Score **15.21**, length **3**: 노래가 너무 평화롭다
- Score **14.67**, length **5**: 너무 배경이랑 노래가 너무 무섭다
- Score **14.31**, length **4**: 메인메뉴 노래가 너무 좋아...
- Score **14.31**, length **4**: 5 만원치고 노래가 너무 없음.

Parameter Results

Setting 1

- $k1 = 1.2$
- $b = 0.75$
- Top 5 documents: **[0, 10017, 99730, 4117, 3295]**
- Average document length: **4**

Setting 2

- $k1 = 2.0$
- $b = 0.75$
- Top 5 documents: **[0, 10017, 99730, 4117, 3295]**
- Average document length: **4**

Setting 3

- $k1 = 1.5$
- $b = 0.0$
- Top 5 documents: **[8365, 99730, 86720, 3295, 4117]**
- Average document length: **13**

Setting 4

- $k1 = 1.5$
- $b = 1.0$
- Top 5 documents: **[0, 10017, 4117, 3295, 99730]**
- Average document length: **4**.

Analysis

When $b = 0$, document length is ignored, and the average length of the Top 5 documents was **13**.

When **b = 1.0**, stronger length normalization was applied, and the average Top 5 document length decreased to **4**.

Therefore, increasing **b** tends to penalize longer documents more strongly.

Changing **k1** from **1.2** to **2.0** while keeping **b = 0.75** did not change the Top 5 ranking in this experiment.

4. Task 3 – Korean Morphological Analysis

Five Korean examples were examined using kiwipiepy.

Case	Error Type	Example	Method
1	Compound noun	서든어택	User dictionary
2	Compound noun	한글패치	User dictionary
3	Neologism	개꿀잼	User dictionary
4	Neologism	갓잼	User dictionary
5	Spacing	텍스트마이닝은재미있다 kiwi.space()	

Case Analysis

Case 1: 서든어택

Before adding the user dictionary:

서든어택 / 표절 / 하 / ㅁ / ㅋㅋㅋㅋㅋ

After adding 서든어택 to the user dictionary, the result remained unchanged because Kiwi already recognized it as a proper noun.

Case 2: 한글패치

Before user-dictionary registration, Kiwi separated the term into:

한글 + 패치

After registering 한글패치, it was recognized as one noun.

Case 3: 개꿀잼

The word was already recognized as a single noun, so adding it to the user dictionary did not substantially change the segmentation.

Case 4: 갓잼

Before registration, 갓잼 was tagged as **NNP**.

After registration, it was tagged as **NNG**.

Case 5: 텍스트마이닝은재미있다

Before spacing correction:

텍스트 / 마 / 이닝 / 은 / 재미있 / 다

After using `kiwi.space()`:

텍스트 마 이닝은 재미있다

However, 마이닝 was still split into 마 and 이닝, showing that spacing correction alone cannot solve every morphological-analysis problem.

Summary

User dictionaries can help preserve compound nouns, neologisms, and domain-specific terms.

However, some words are already correctly recognized by Kiwi, so adding them to a user dictionary may not change the result.

Spacing correction can improve preprocessing, but user-dictionary handling may still be necessary for domain-specific expressions.

5. Bonus Task – N-gram Smoothing

The dataset was divided into training and test data.

There were **106,610 evaluation bigrams**, and **86,663** of them were unseen in the training data.

Therefore, the unseen bigram rate was **81.3%**, demonstrating the zero-probability problem of maximum-likelihood n-gram models.

Add-k Smoothing Results

- Add-1.0 PPL = **188,014**
- Add-0.1 PPL = **131,125**
- Add-0.01 PPL = **94,395**

Kneser-Ney Result

- Discount D = **0.75**
- Kneser-Ney PPL = **23,338**

The terms with high continuation probability included:

게임, 이, 너무, 그냥, 좀, 진짜, 정말, 다, 더, 왜.

Analysis

Kneser-Ney smoothing achieved much lower perplexity than the tested Add-k methods.

The best Add-k result was **94,395**, whereas Kneser-Ney achieved **23,338**.

This suggests that Kneser-Ney redistributes probability mass to unseen n-grams more effectively than simple Add-k smoothing.

6. Conclusion

In this assignment, Count, TF-IDF, BM25, Korean morphological analysis, and n-gram smoothing were examined.

The TF-IDF experiment showed that rare words receive higher IDF values and can therefore receive greater importance.

The BM25 experiment showed that the b parameter controls document-length normalization. When b increased from **0** to **1.0**, the average length of the Top 5 documents decreased from **13** to **4**.

The Korean morphological-analysis experiment showed that user dictionaries can improve the treatment of compound nouns and domain-specific expressions, while spacing correction is useful but not sufficient for every case.

Finally, Kneser-Ney smoothing achieved a perplexity of **23,338**, substantially lower than the tested Add- k methods.

GitHub Repository:

<https://github.com/LYT-213/text-mining-week2-assignment>